

Uniwersytet Radomski im. Kazimierza Pułaskiego w Radomiu	
Wydział Transportu, Elektrotechniki i Informatyki	
Techniczne zastosowania sieci neuronowych	
Rok akademicki 2025/2026	
Projekt: 3	
Autorzy:	Ivan Pohodaev Andrii Yevlakh Maksymilian Wieczorek

Cel

Zastosowanie rekurencyjnej sieci neuronowej Hopfielda do odtwarzania obrazów (P,R,O)

1. Struktura sieci neuronowej

W projekcie wykorzystano jednowarstwową, rekurencyjną sieć Hopfielda (dyskretną sieć autoasocjacyjną) z pełnym sprzężeniem zwrotnym. Oznacza to, że każdy neuron łączy się z każdym innym, z wyłączeniem sprzężeń do samego siebie. Ponieważ analizowane wzorce wprowadzane są z poziomu siatki 10x10 pikseli, sieć posiada 100 wejść oraz jedną warstwę składającą się ze 100 neuronów. Połączenia między nimi definiuje symetryczna macierz wag W o wymiarach 100 X 100, mająca zera na głównej przekątnej ($W_{ii} = 0$). Jako funkcję aktywacji zastosowano dwubiegunową funkcję skokową. Nowy stan i -tego neuronu ($x_i \in \{-1, 1\}$) ustala się na podstawie sumy ważonej jego sygnałów wejściowych (s_i), do której wstrzykiwany jest minimalny szum losowy w celu uniknięcia minimów lokalnych. Zależność tę określa wzór (1):

$$x_i = \begin{cases} 1, & \text{dla } s_i \geq 0 \\ -1, & \text{dla } s_i < 0 \end{cases} \quad (1)$$

gdzie:

x_i – stan i -tego neuronu

s_i – suma ważona wejść

2. Opis algorytmu uczenia

Do wyuczenia sieci zastosowałem algorytm oparty na regule pseudoodwrotności. Wybrałem ten sposób zamiast zwykłej reguły Hebb'a, bo dzięki temu sieć dużo lepiej radzi sobie z zapamiętywaniem wzorców, które są do siebie trochę podobne (nie są ortogonalne), co znacznie zwiększa jej pojemność. Proces uczenia odbywa się tu jedno krokowo (nie ma żadnych epok). W kodzie działa to tak, że program bierze spłaszczone wzorce naszych liter (P, R, O) i tworzy z nich jedną dużą macierz (zmienna $\mathbf{M}$). Potem program wylicza z niej tzw. macierz Grama ($\mathbf{G}$), odwraca ją i przez odpowiednie przemnożenie tych macierzy generuje gotową macierz wag $\mathbf{W}$ dla całej sieci.

Modyfikacje: Kluczową modyfikacją w kodzie jest sztuczne wyzerowanie głównej przekątnej wyliczonej macierzy wag (użyłem do tego funkcji). Zrobiłem to po to, żeby usunąć połączenia neuronów z samymi sobą (auto-połączenia). Bez tego kroku sieć Hopfielda mogłaby się zacinać na błędnych rozwiązaniach, zamiast dążyć do właściwego wzorca. [1,3]

3. Ciąg uczący (CU)

Mój ciąg uczący to po prostu tablice reprezentujące narysowane kształty. Ponieważ siatka do rysowania w programie ma wymiary 10x10, każdy wzorec po spłaszczeniu składa się dokładnie ze 100 elementów (wektorów jednowymiarowych). Z racji tego, że sieć Hopfielda działa na zasadzie autoasocjacji, zbiór ten nie zawiera żadnych dodatkowych "wartości zadanych" (etykiet czy klas). Dla tej sieci wprowadzany obraz jest jednocześnie celem samym w sobie – sieć ma go po prostu zapamiętać i odtworzyć. Do reprezentacji danych zastosowałem kodowanie dwubiegunowe. Oznacza to, że każdy piksel zamalowany (aktywny) w interfejsie otrzymuje wartość 1, natomiast puste tło (nieaktywne komórki) kodowane jest jako -1. W programie zdefiniowałem domyślnie trzy elementy ciągu uczącego (litery P, R oraz O). Zgodnie z wytycznymi, poniżej wklejam fragment kodu pokazujący, jak te trzy wzorce są zakodowane w programie:[1,3]

```
12
13 WZORZEC_P = np.array(
14     [
15         [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1],
16         [-1, -1, -1, 1, -1, -1, 1, -1, -1, -1],
17         [-1, -1, -1, 1, -1, -1, 1, -1, -1, -1],
18         [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1],
19         [-1, -1, -1, 1, -1, -1, -1, -1, -1, -1],
20         [-1, -1, -1, 1, -1, -1, -1, -1, -1, -1],
21         [-1, -1, -1, 1, -1, -1, -1, -1, -1, -1],
22         [-1, -1, -1, 1, -1, -1, -1, -1, -1, -1],
23         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
24         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
25     ]
26 )
27
28 WZORZEC_R = np.array(
29     [
30         [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1],
31         [-1, -1, -1, 1, -1, -1, 1, -1, -1, -1],
32         [-1, -1, -1, 1, -1, -1, 1, -1, -1, -1],
33         [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1],
34         [-1, -1, -1, 1, -1, 1, -1, -1, -1, -1],
35         [-1, -1, -1, 1, -1, -1, 1, -1, -1, -1],
36         [-1, -1, -1, 1, -1, -1, 1, -1, -1, -1],
37         [-1, -1, -1, 1, -1, -1, -1, -1, 1, -1],
38         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
39         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
40     ]
41 )
42
43 WZORZEC_O = np.array(
44     [
45         [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1],
46         [-1, -1, 1, -1, -1, -1, 1, -1, -1, -1],
47         [-1, -1, 1, -1, -1, -1, 1, -1, -1, -1],
48         [-1, -1, 1, -1, -1, -1, 1, -1, -1, -1],
49         [-1, -1, 1, -1, -1, -1, 1, -1, -1, -1],
50         [-1, -1, 1, -1, -1, -1, 1, -1, -1, -1],
51         [-1, -1, -1, 1, 1, 1, -1, -1, -1, -1],
52         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
53         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
54         [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1],
55     ]
56 )
57
```

Rys.1. Wzorce są zakodowane w programie

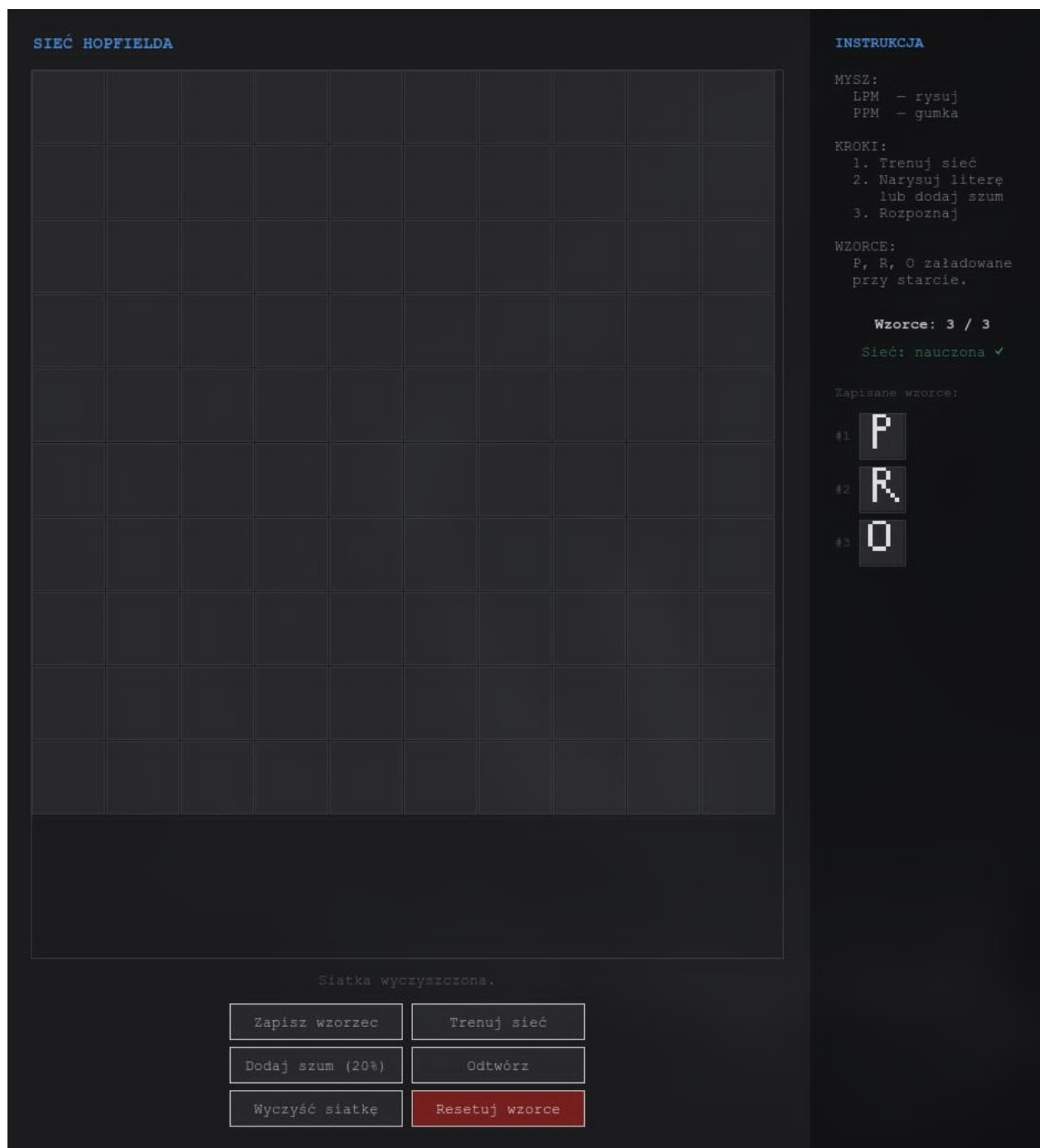
4. Ciąg testowy (CT)

W moim programie ciąg testowy nie jest wpisany na sztywno w kodzie tak jak wzorce uczące. Tworzy się go dynamicznie, na bieżąco podczas korzystania z aplikacji. Sposób kodowania jest identyczny jak w ciągu uczącym (1 dla zamalowanego piksela, -1 dla pustego tła). Tutaj również nie ma żadnych przypisanych "wartości zadanych" czy etykiet – wprowadzony wektor testowy trafia na wejście, a sieć ma za zadanie ustabilizować się i "przypomnieć" sobie, do jakiego wyuczonego obrazu (P, R lub O) jest mu najbliższej. Do generowania elementów ciągu testowego używam w programie dwóch opcji:

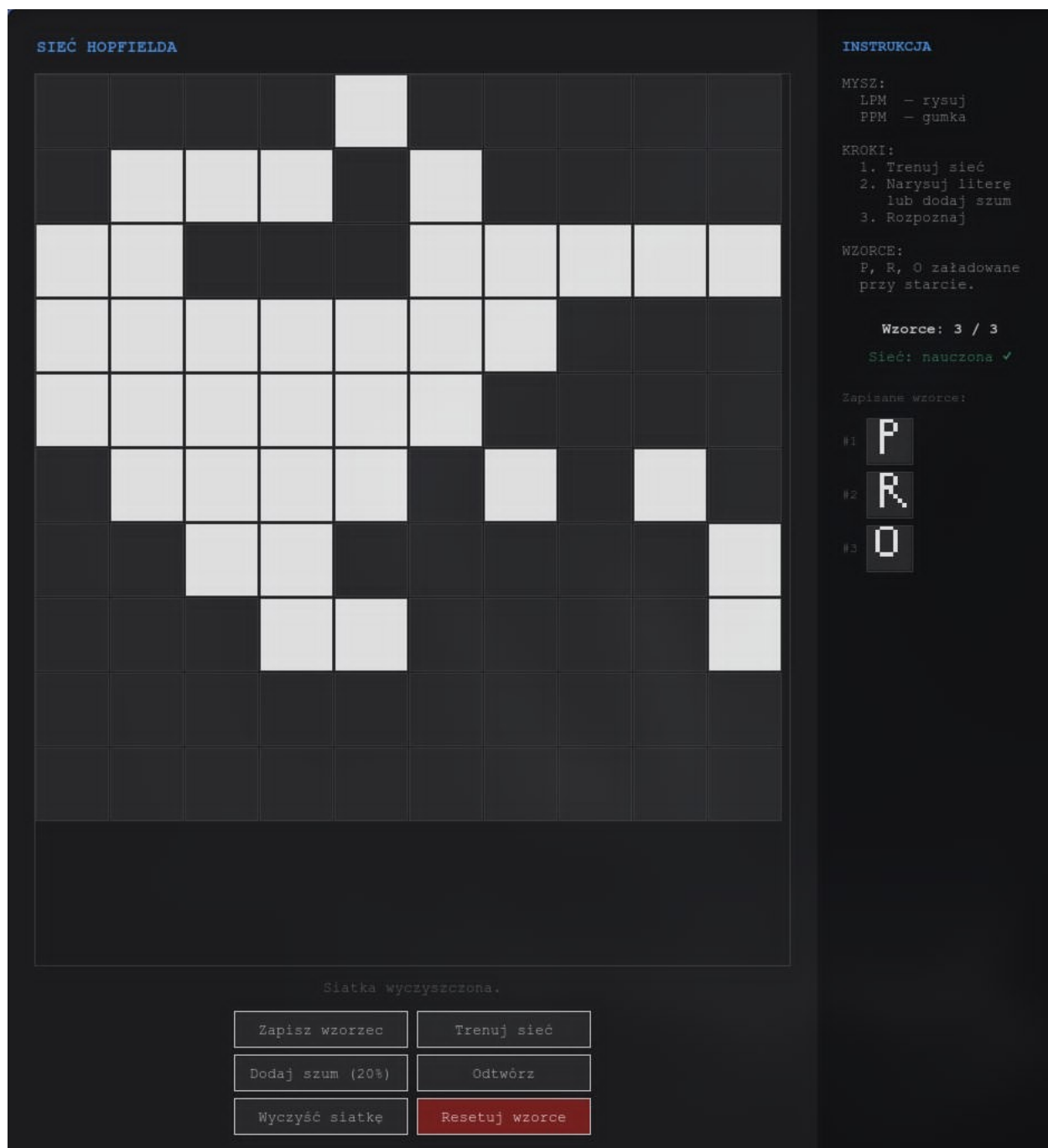
- **Ręczne wprowadzanie:** Rysowanie kursorem dowolnego kształtu (np. niepełnej litery) na siatce 10x10.
- **Dodawanie szumu:** Użycie przycisku "Dodaj szum (20%)", który odpala funkcję `_dodaj_szum`. Funkcja ta przelatuje przez siatkę i z 20-procentowym prawdopodobieństwem odwraca stan każdego piksela (zmienia 1 na -1 i odwrotnie).[1,3]

5. Implementacja

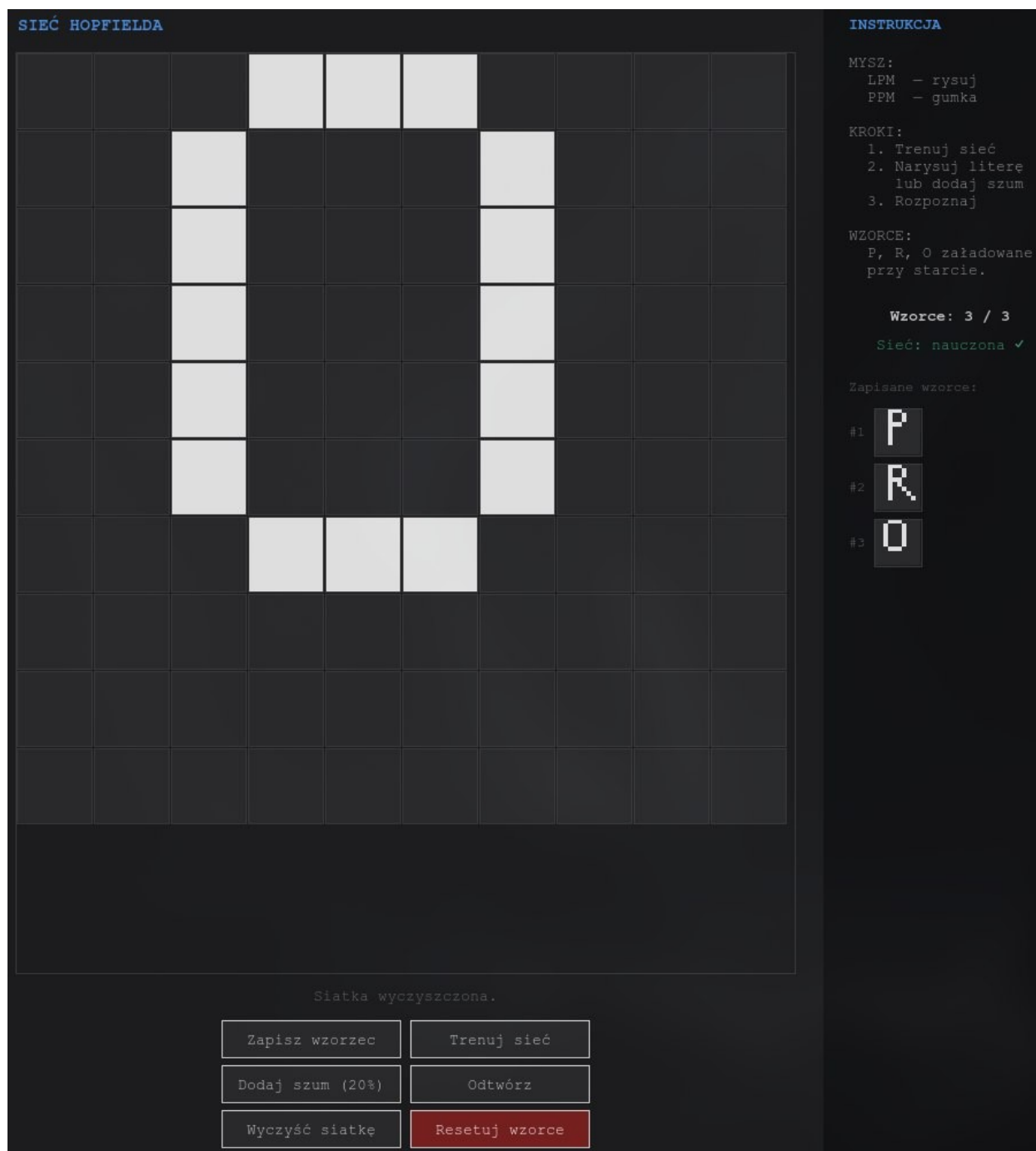
Program napisaliśmy w języku Python. Do stworzenia interfejsu graficznego (GUI) wykorzystaliśmy wbudowaną bibliotekę `tkinter`, a wszystkie obliczenia matematyczne (mnożenie macierzy, odwracanie) realizuje biblioteka `numpy`. Wygląd okna aplikacji zaraz po uruchomieniu prezentuje (Rys. 2). Interfejs jest czytelnie podzielony na dwie części: po lewej stronie znajduje się główna, interaktywna siatka 10x10 do rysowania oraz przyciski sterujące, natomiast po prawej umieściliśmy boczny panel zawierający krótką instrukcję obsługi, aktualny status sieci oraz miniaturki domyślnie zapamiętanych wzorców (liter P, R, O). Sam proces działania algorytmu w praktyce zobrazowaliśmy na kolejnych zrzutach ekranu. Na (Rys. 3) przedstawiono stan interfejsu w momencie przygotowania do testów – na siatce widać mocno za szumiony, uszkodzony wzorec przed uruchomieniem procesu rozpoznawania. Z kolei (Rys. 4) ukazuje ostateczny wynik działania naszej aplikacji. Widać na nim, że sieć Hopfielda pomyślnie zaktualizowała stany neuronów i poprawnie zrekonstruowała z tego szumu idealny kształt litery "R".[2]



Rys. 2. Interfejs główny programu po uruchomieniu
Źródło: opracowanie własne



Rys. 3. Za szumiony wzorzec przed rozpoznaniem
Źródło: opracowanie własne



Rys. 4. Wynik działania sieci
Źródło: opracowanie własne

Instrukcja obsługi programu:

1. **Rysowanie/Usuwanie:** Kliknij i przytrzymaj Lewy Przycisk Myszy (LPM) na siatce, aby zamalować piksele. Użyj Prawego Przycisku Myszy (PPM), aby je zmazać (użyć gumki).
2. **Przygotowanie do pracy:** Po uruchomieniu programu, litery P, R i O są już w pamięci. Zanim sieć coś rozpozna, musisz kliknąć przycisk "**Trenuj sieć**". Etykieta po prawej stronie zmieni się na "nauczona".
3. **Testowanie (Odtwarzanie):**
 - Narysuj na siatce uszkodzony kształt zapamiętanej litery.

- Alternatywnie, narysuj idealną literę i kliknij **"Dodaj szum (20%)"**, co wprowadzi losowe błędy.
 - Kliknij **"Odtwórz"**. Program na żywo pokaże proces asynchronicznej aktualizacji stanów neuronów, aż obraz ustabilizuje się do najbliższego zapamiętanego wzorca.
4. **Własne wzorce:** Możesz wyczyścić siatkę ("Wyczyść siatkę"), narysować swój symbol i kliknąć "Zapisz wzorec". Pamiętaj, aby potem ponownie kliknąć "Trenuj sieć". "Resetuj wzorce" usuwa wszystko z pamięci.

5.1. Opis najważniejszych fragmentów kodu

Ten fragment kodu odpowiada za jednorazowe wyliczenie wag sieci dla wszystkich wprowadzonych wzorców. Wykorzystana tu jest reguła pseudoodwrotności (szybkie operacje na macierzach za pomocą biblioteki numpy). Kluczową liniijką jest tutaj `np.fill_diagonal(self.W, 0)`, która na samym końcu obliczeń wymusza zera na głównej przekątnej macierzy wag. Służy to wyeliminowaniu autopołączeń (neuron nie może być połączony sam ze sobą), co jest niezbędne do poprawnego działania sieci Hopfielda.[2]

```

67
68     def naucz(self, wzorce):
69         n = self.rozmiar
70         p = len(wzorce)
71         M = np.array(wzorce)
72         G = M @ M.T
73         G_inv = np.linalg.inv(G)
74         self.W = M.T @ G_inv @ M
75         np.fill_diagonal(self.W, 0)
76         self.nauczona = True
77

```

Rys. 5. Funkcja ucząca sieć
Źródło: opracowanie własne

Poniższy fragment kodu to główna pętla odpowiedzialna za "przypominanie" sobie kształtu przez sieć. Aktualizacja działa w trybie asynchronicznym – w każdym kroku (`for _ in range(40)`) program losuje kolejność odpytywania neuronów (`np.random.permutation`). Dla każdego neuronu liczona jest suma ważona (`np.dot`), do której celowo dodawany jest mikroskopijny szum losowy (pomaga to sieci wyskoczyć z tzw. lokalnych minimów energetycznych). Nowa wartość jest wyznaczana przez prostą funkcję skokową. Dzięki dodaniu funkcji `time.sleep(0.03)` użytkownik widzi w programie płynną animację "naprawiania" obrazu. Pętla przerywa działanie (`break`), gdy sieć się ustabilizuje, czyli gdy nowy stan obrazu jest identyczny ze starym. [2]

```

428
429     for _ in range(40):
430         stary_x = x.copy()
431
432         kolejnosc = np.random.permutation(len(x))
433
434         for i in kolejnosc:
435             s = np.dot(self.siec.W[i], x) + (random.random() * 0.02 - 0.01)
436
437             nowy = 1.0 if s >= 0 else -1.0
438
439             x[i] = nowy
440
441             # обновление GUI
442             self.siatka = x.reshape((ROZMIAR_SIATKI, ROZMIAR_SIATKI))
443             self._odrysuj_siatke()
444
445             self.root.update()
446
447             # скорость анимации
448             time.sleep(0.03)
449
450         if np.array_equal(x, stary_x):
451             break
452
453

```

Rys. 6. Asynchroniczna aktualizacja stanów

Źródło: opracowanie własne

Poniższy fragment kodu służy do celowego zniekształcania obrazu, co jest bardzo przydatne podczas testowania skuteczności sieci. Program używa dwóch pętli, aby przejść przez każdą komórkę (piksel) na naszej siatce. Dla każdego piksela losuje liczbę – jeśli jest ona mniejsza niż 0.20 (co daje 20% szans), stan tego piksela zostaje odwrócony na przeciwny (matematycznie: pomnożony przez -1). W ten sposób tworzymy za szumiony wzorec testowy.[2]

```

403
404     def _dodaj_szum(self):
405         ile = 0
406         for i in range(ROZMIAR_SIATKI):
407             for j in range(ROZMIAR_SIATKI):
408                 if random.random() < 0.20:
409                     self.siatka[i, j] *= -1
410                     ile += 1
411             self._odrysuj_siatke()
412             self._ustaw_status(f"Szum dodany ({ile} pikseli odwrócono).", KOL TEKST)
413

```

Rys. 7. Funkcja dodająca szum

Źródło: opracowanie własne

Ten poniższy fragment kodu to metody odpowiadają za interaktywność interfejsu graficznego. Gdy użytkownik klika na ekranie, program odbiera dokładne współrzędne kursora w pikselach (np. oś X i Y). Metoda `_komorka_z_eventu` korzysta z dzielenia całkowitego (`//`), aby przeliczyć te piksele na konkretne indeksy naszej macierzy 10x10. Gdy program wie już, w którą "kratkę" kliknęliśmy, funkcja `_start_rysowania` zamienia wartość tej komórki na 1.0 (aktywny piksel) i odświeża wygląd siatki na ekranie.[2]


```

331 # -----
332 def _komorka_z_eventu(self, event):
333     j = event.x // ROZMIAR_KOMORKI
334     i = event.y // ROZMIAR_KOMORKI
335     if 0 <= i < ROZMIAR_SIATKI and 0 <= j < ROZMIAR_SIATKI:
336         return i, j
337     return None
338
339 def _start_rysowania(self, event):
340     self.rysowanie = True
341     kom = self._komorka_z_eventu(event)
342     if kom:
343         self.siatka[kom[0], kom[1]] = 1.0
344         self._odrysuj_siatke()
345

```

Rys. 8. Konwersja kliknięcia myszką na współrzędne siatki
Źródło: opracowanie własne

6. Proces uczenia

W przypadku mojej sieci Hopfielda proces uczenia wygląda zupełnie inaczej niż w standardowych sieciach (takich jak wielowarstwowe perceptrony), dlatego ciężko go opisać dokładnie według narzuconego schematu z instrukcji. Zastosowałem algorytm oparty na regule pseudo odwrotności, co oznacza, że uczenie jest tutaj jednokrokowe i w pełni analityczne. Sieć nie musi wielokrotnie "ogłądać" wzorców i powoli korygować błędów, przez co czas uczenia wynosi u mnie dokładnie zero epok. Cały proces to po prostu jednorazowe wykonanie kilku szybkich operacji macierzowych w bibliotece numpy (mnożenie i odwracanie macierzy Grama), co zajmuje programowi dosłownie ułamek sekundy. Z tego samego powodu w moim kodzie nie występują żadne typowe parametry, takie jak współczynnik szybkości uczenia (learning rate), dopuszczalny błąd czy momentum – wagi są wyliczane "na sztywno" z gotowego wzoru matematycznego. Skoro proces nie opiera się na pętli, to nie ma też klasycznego warunku zakończenia uczenia. Algorytm po prostu wylicza macierz wag, sztucznie wymusza zera na jej głównej przekątnej (żeby usunąć niechciane auto połączenia neuronów) i w tym momencie działanie metody się kończy, a w interfejsie pojawia się informacja, że sieć jest nauczona i gotowa do testów. [4]

7. Testowanie

Eksperymentalną weryfikację skuteczności oraz odporności zaimplementowanego modelu przeprowadzono poprzez serię testów z wykorzystaniem interfejsu graficznego aplikacji. Procedura badawcza polegała na wprowadzeniu wyuczonych wzorców liter, poddaniu ich procesowi intencjonalnego zniekształcania za pomocą algorytmu wprowadzania szumu losowego (na poziomie 20%), a następnie uruchomieniu procedury rekonstrukcji asynchronicznej. Na podstawie przeprowadzonych prób stwierdzono, że sieć wykazuje wysoką skuteczność w przypadku umiarkowanego zaszumienia sygnału wejściowego. Proces wizualizacji dynamicznej pozwolił zaobserwować płynną konwergencję struktury i skuteczne usunięcie wprowadzonych zaburzeń, co doprowadziło do odtworzenia idealnego, zapamiętanego obrazu litery „R”. Przeprowadzona analiza wykazała jednak ograniczenia architektury w warunkach ekstremalnego zniekształcania obrazu wejściowego (np. przy ręcznym zamazaniu ponad 50% powierzchni matrycy). W takich przypadkach sieć traciła zdolność poprawnej asocjacji, stabilizując się w fałszywych stanach stacjonarnych (minimach lokalnych powierzchni energii), co skutkowało generowaniem zniekształconych, niepoprawnych struktur nieodpowiadających żadnej z wyuczonych klas [4].

8. Wnioski

Na podstawie przeprowadzonych testów i analizy działania zaimplementowanego programu można stwierdzić, że sieć Hopfielda poprawnie realizuje zadanie, sprawnie zapamiętując zadane wzorce graficzne w postaci liter P, R oraz O. Model wykazuje odpowiednią odporność na zakłócenia sygnału, potrafiąc prawidłowo zrekonstruować wyuczone litery nawet w przypadku obrazów zniekształconych poprzez wprowadzenie 20-procentowego szumu losowego. Istotną zaletą zaimplementowanego rozwiązania jest zoptymalizowany sposób uczenia, który dzięki wykorzystaniu szybkich obliczeń matematycznych realizowany jest błyskawicznie w jednym kroku, co eliminuje potrzebę długotrwałego i żmudnego trenowania algorytmu. Zauważalną wadą programu jest jednak ograniczona pojemność pamięciowa oraz podatność na znaczne zniekształcenia sygnału wejściowego. W przypadku zbyt silnego zamazania lub uszkodzenia obrazu aplikacja całkowicie się gubi i stabilizuje w minimach lokalnych, generując losowe, bezsensowne kształty zamiast poprawnej, oczekiwanej litery. Pomimo tych architektonicznych ograniczeń, stworzona sieć neuronowa bardzo dobrze sprawdza się w swoim podstawowym zastosowaniu, jakim jest proste korygowanie i odtwarzanie wprowadzanych obrazów obarczonych niewielkimi błędami.

9. Bibliografia

[1] Kriesel D., A Brief Introduction to Neural Networks, Wydanie elektroniczne (darmowy skrypt PDF udostępniany przez autora), Bonn 2007.

[2] NumPy v1.26 Manual, URL: <https://numpy.org/doc/stable/>, data ostatniego dostępu 27.05.2026.

[3] Sieć Hopfielda, Wikipedia, wolna encyklopedia, URL: https://pl.wikipedia.org/wiki/Sieć_Hopfielda, data ostatniego dostępu: 01.06.2026.

[4] Tadeusiewicz R., Sieci neuronowe, Akademicka Oficyna Wydawnicza RM, Warszawa 1993 (publikacja udostępniona jako darmowy PDF przez autora).

WKŁAD DO PROJEKTU		
Ivan Pohodaev	Andrii Yevlakh	Maksymilia Wieczorek
Wersja kodu: Poprawka i pisanie (aktualnego kodu) Edycja sprawozdania	Wersje kodu: Pisanie początku kodu Edycja sprawozdania	Wersje kodu Małe poprawki poprawki kodu Stworzenie sprawozdania i pisanie notatek